5 Static authorization models

This chapter covers

- Distinguishing static versus dynamic authorization models
- Managing permissions with Access Control Lists (ACLs)
- Assigning access through Role-Based Access Control (RBAC)
- Mapping ACLs and RBAC to the reference architecture components
- Evaluating the strengths and limitations of static models
- Identifying where ACLs and RBAC still make sense in modern systems

When ACME Corp launched its first SaaS product, authorization was an afterthought. With a handful of developers and a few early customers, it was easy to hardcode access to the admin dashboard and restrict sensitive features to known team members. Everyone had access to staging environments, and customer data was stored in separate folders, protected more by convention than by control.

As the company grew—from a scrappy startup to a fast-scaling enterprise—its access control needs became more complex. Contractors needed limited access to customer support tools. Sales reps required region-based access to accounts. Engineering teams asked for staging environments with access scoped by project. Support teams needed the ability to debug customer issues without compromising security. And Finance needed a better understanding of access for regulatory compliance.

In the absence of a structured system, developers responded the only way they could: by adding custom if-then conditionals. These conditionals crept into APIs, databases, and front-end logic. Some were based on email domains, others on team assignments in a database, and still others on temporary configuration flags.

What started as a few simple checks grew into a sprawling web of undocumented logic. Access decisions varied across services, and exceptions